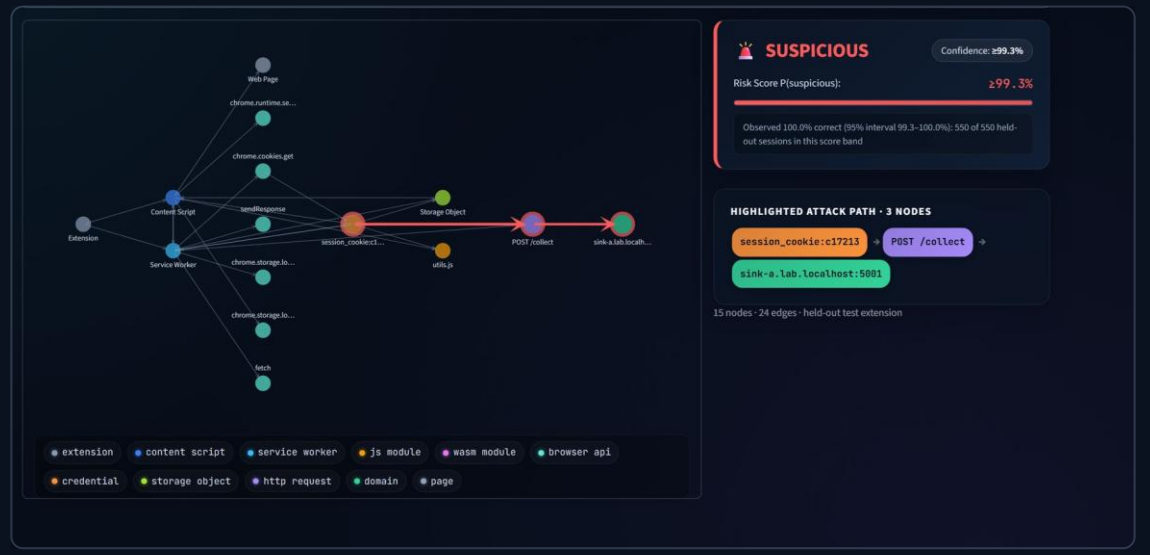
AuthGraph

Authentication-aware graph learning to detect unauthorized credential flows in browser extensions

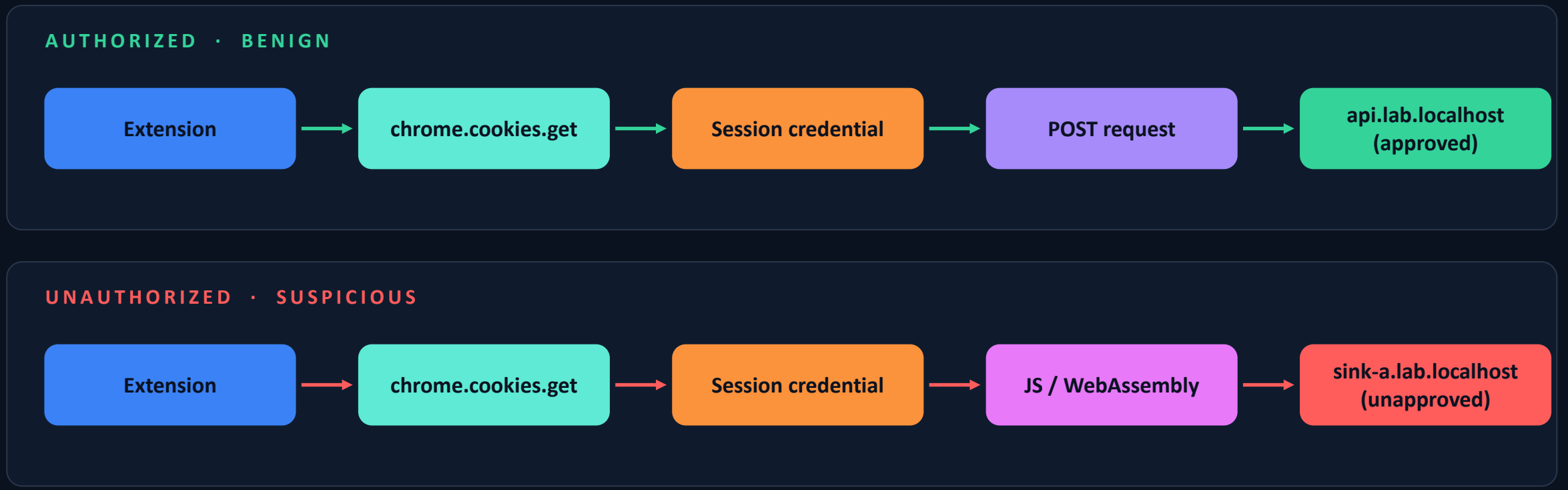


Controlled-lab prototype · real Chromium · 576 sessions · 144 extensions

Presented by Neranjana

A session cookie is one API call away

Extensions legitimately read cookies, storage and pages. The question is where the credential goes, not whether it is read.



Same API calls, same credential. Only the destination and the data flow differ, so API-level rules are not enough.

What we found, in four numbers

576

real browser sessions from 144 generated MV3 extensions in 12 scenarios; 0 credential leaks

0.929

GraphSAGE recall on Wasm flows never seen in training. Taint rule 0.646, flat models 0.62–0.69

0.450

R-GCN, our proposed heterogeneous model, on the same test. It lost, and we report it

≥99.3%

top-band confidence: the 95% lower bound of accuracy observed on held-out data. Never 100%

THE MESSAGE

Graph structure helps exactly where instrumentation is incomplete: when taint is lost inside WebAssembly, the graph still exposes the flow.

But the relation-typed R-GCN did not beat plain GraphSAGE on this small synthetic dataset. The honest result is the useful one.

Positioning: a narrow claim on top of strong prior work

PRIOR WORK (CHECKED BY SEARCH)

- DoubleX (ACM CCS 2021): static Extension Dependence Graph for vulnerable data flows; 154,484 Chrome extensions.
- Arcanum (USENIX Security 2024): dynamic taint tracking of page content to network sinks in modern extensions.
- Heterogeneous-graph GNNs for malware behaviour graphs already exist (e.g. MalSage; title seen in search only).
- Wasm–JS collaborative attacks on Chrome extensions (Electronics; title seen, not yet read in full).

OUR CLAIM

An authentication-aware heterogeneous graph of runtime behaviour, with GNN detection of credential flows, compared with rules, flat ML and homogeneous GNNs under extension-level and unseen-flow splits.

WE DO NOT CLAIM

- the first extension graph or detector
- the first GNN in browser security
- any result on real-world extensions (NOT YET MEASURED)

A safe lab: nothing leaves localhost



Localhost only

App on :5000, collector and sink hosts on *.lab.localhost:5001 and 127.0.0.1. No external network.



Synthetic credentials

Fresh random cookie and JWT per login. Telemetry stores only a hashed ID, never a value.



Controlled behaviours

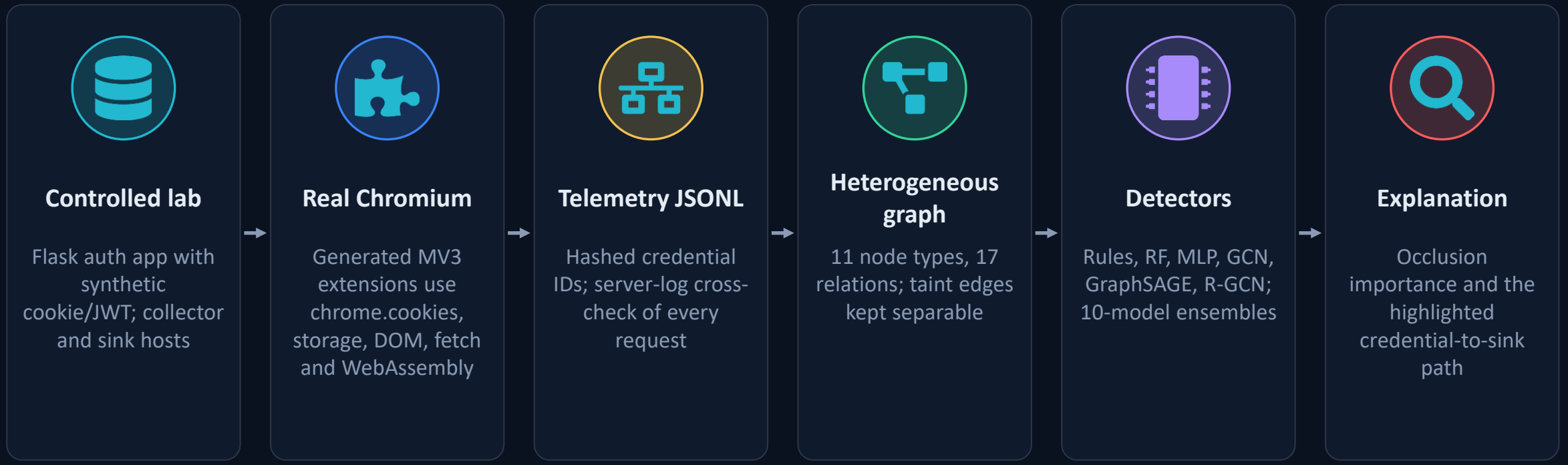
Proof-of-concept extensions exist only to produce labelled telemetry. No persistence, no real malware.



Label by design

Suspicious = credential-derived data sent to an unapproved destination. Fixed by scenario, not by a detector.

System: from a real browser event to an explained verdict



Everything on localhost · synthetic credentials only · graph + verdict + path shown live in the dashboard

Dataset: 576 real sessions, hard negatives included

576
sessions

144
extensions

288 / 288
benign / suspicious

7,258
events captured

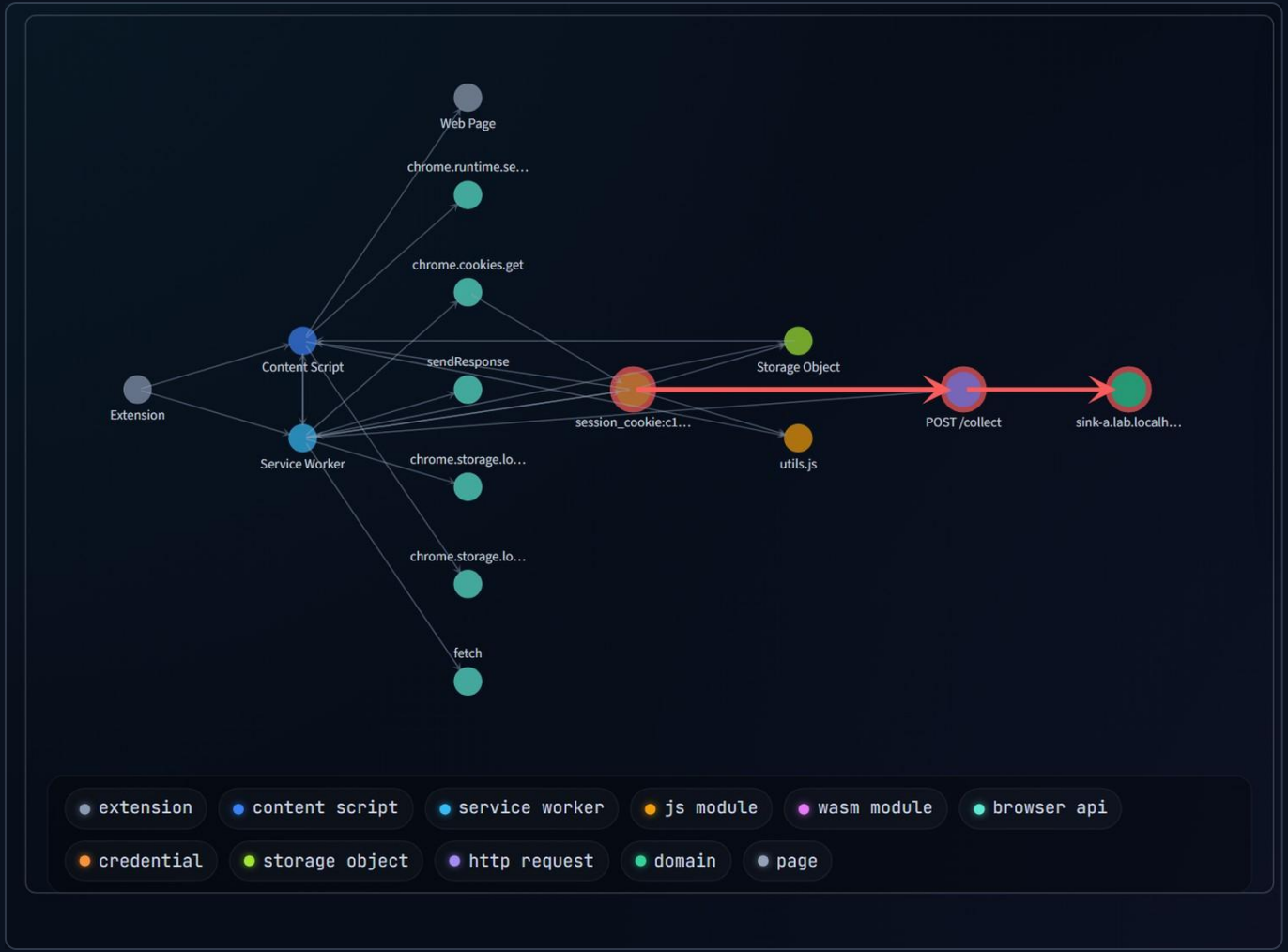
INTEGRITY CHECKS (EVERY RUN)

- **0** credential-value leaks in any event
- **0** events failing the schema
- **0** duplicate events
- **576 / 576** sessions where telemetry matches the server log

B1	DOM only
B2	storage settings
B3	approved API
B4	authenticated call to approved API (hard negative)
B5	credential read + unrelated analytics (hard negative)
B6	benign WebAssembly
S1	direct credential to sink
S2	via 1–3 JS hops
S3	via WebAssembly
S4	content script ↔ service worker
S5	staged through storage
S6	chunked to a raw-IP sink

Wasm output loses its taint tag with probability 0.5 (a design choice modelling a JS-level tracker). Result: 254 suspicious sessions have taint visible at the sink, 34 have it lost.

An authentication-aware graph of each session



REPRESENTATION

- 11 node types: extension, content script, service worker, JS, Wasm, API, credential, storage, request, domain, page
- 17 relations, including flows_cred and carries (taint) that can be ablated
- Mean 13.2 nodes and 19.4 edges per session
- No names or scenario labels used as features

Experimental protocol: built to avoid fooling ourselves

DETECTORS

- Rule R1: credential accessed AND unapproved request
- Rule R2: unapproved request carrying a credential taint tag
- Random Forest and MLP on count features
- GCN, GraphSAGE, R-GCN in plain PyTorch
- 10-model ensembles for the dashboard scores

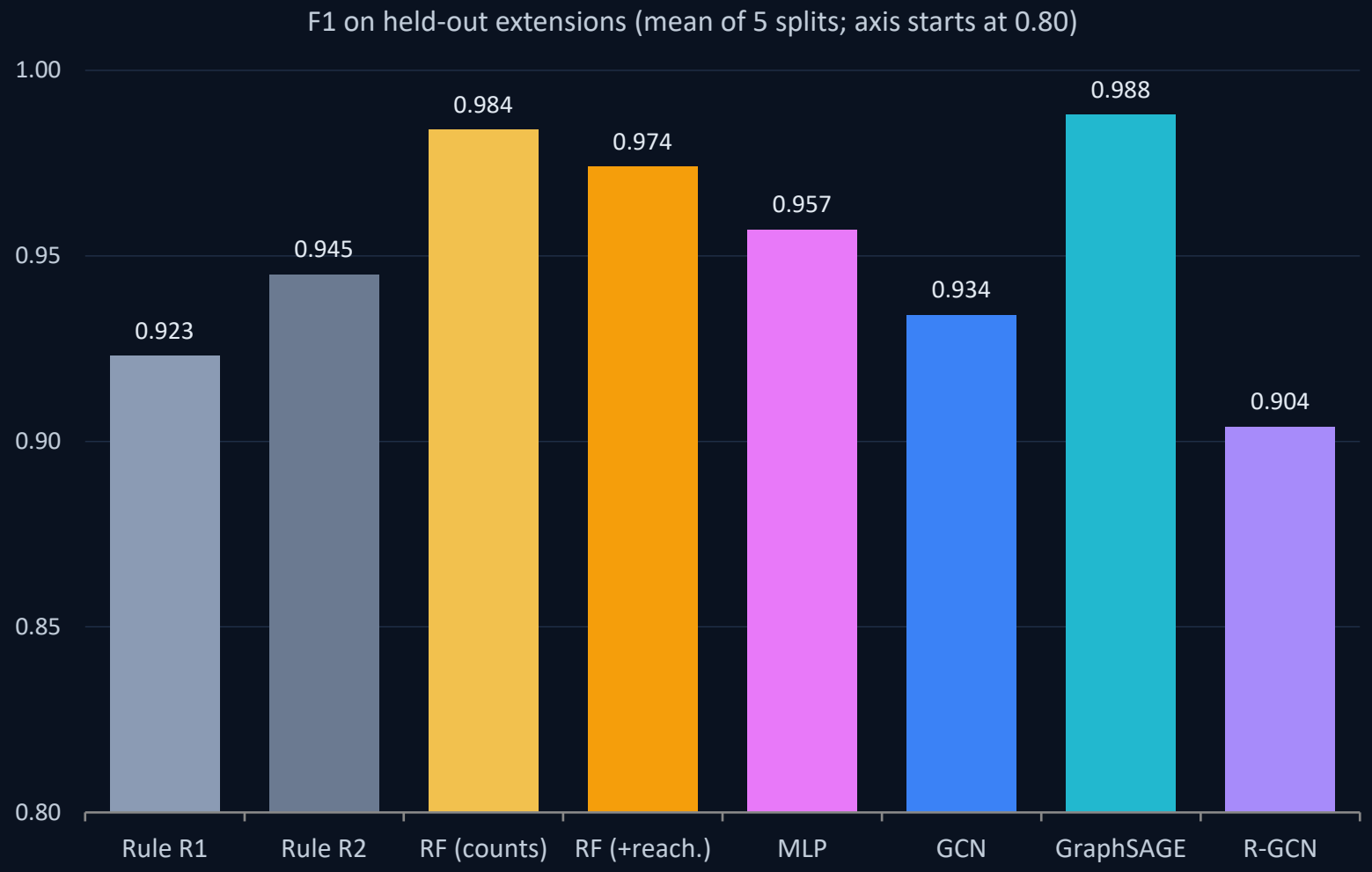
SPLITS

- Extension-level: all sessions of an extension stay in one split
- 5 split seeds, mean ± std
- Unseen flow: hold out Wasm families (S3 + S5) entirely
- Leave-one-family-out for S1 to S6

RIGOR

- Validation-only tuning, identical grid for all GNNs
- Test sets never used for any choice
- Threshold 0.5, not tuned
- Every run logged; the untuned v1 run is kept
- NumPy Random Forest (a local policy blocked sklearn's tree module)

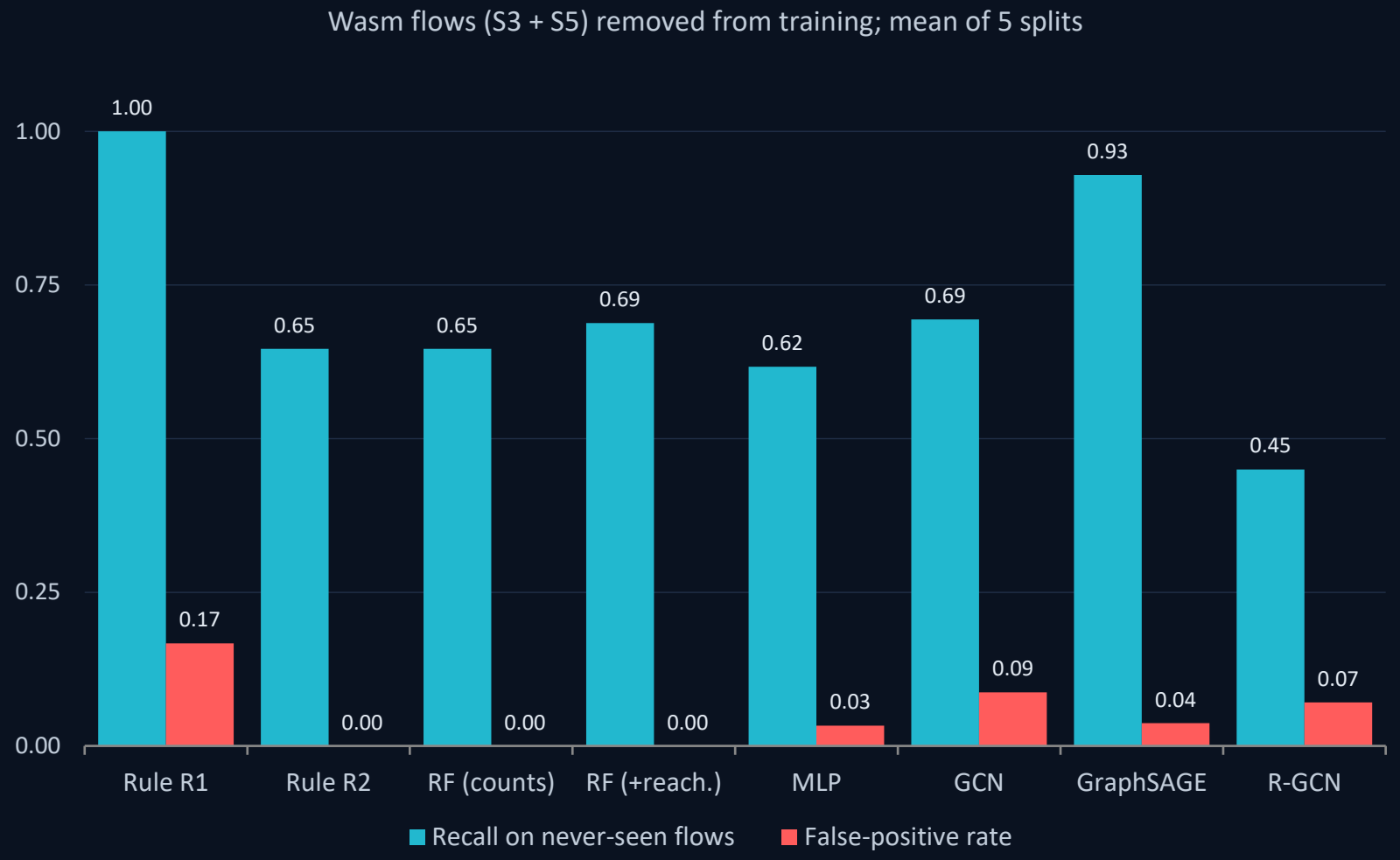
Standard split: easy when taint is visible



HOW TO READ THIS

- RF and MLP on simple counts already reach F1 0.98 and 0.96
- GraphSAGE is best: F1 0.988, ROC-AUC 1.000
- Rule R1 flags 16.7% of benign sessions
- R-GCN is lowest of the learned models: 0.904
- This split does not separate the methods, so the unseen-flow test matters

Never-seen Wasm flows: graph structure still detects them



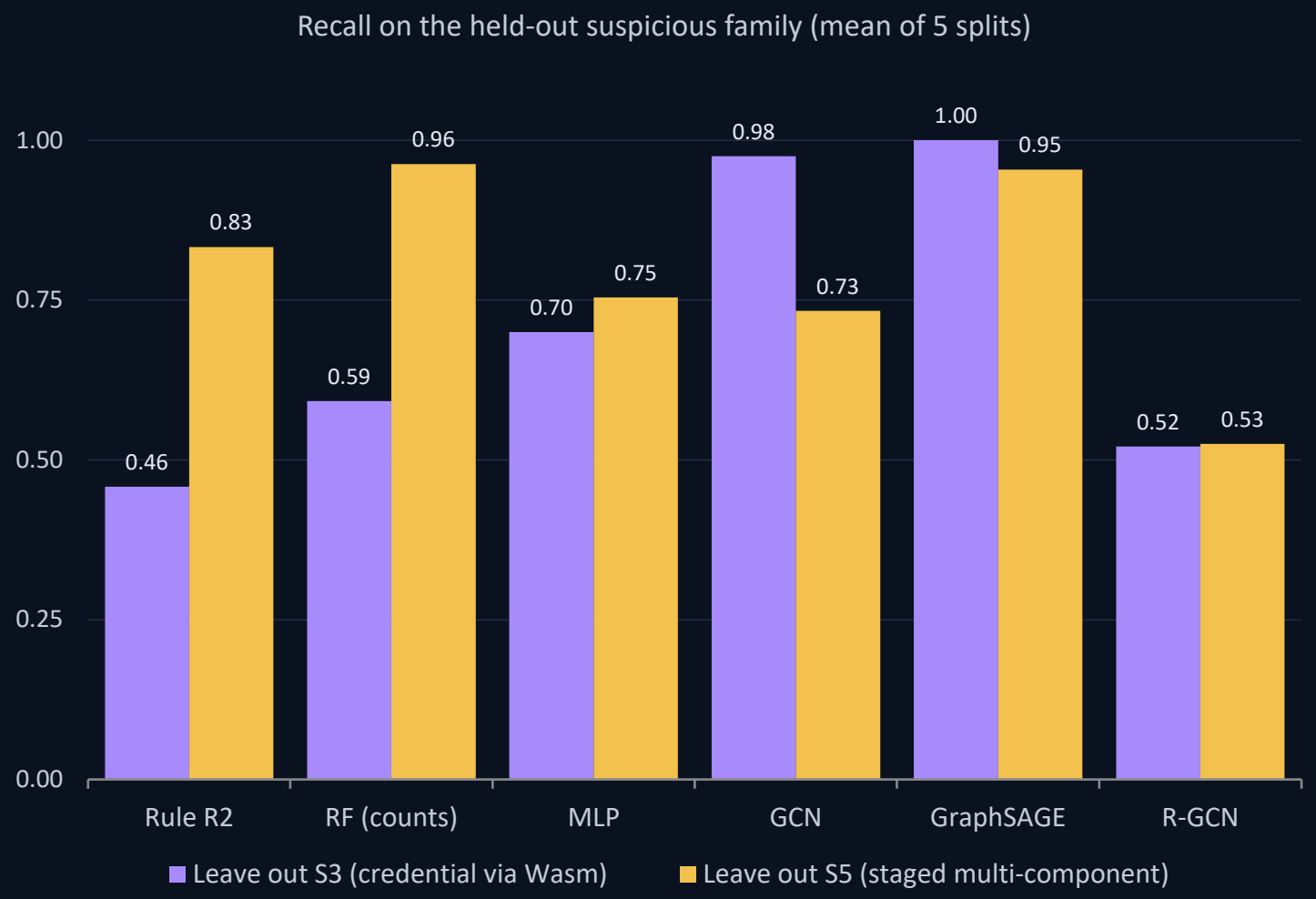
THE RESEARCH MOMENT

0.929

GraphSAGE recall at 3.8% false positives

- Taint rule R2: 0.646, because taint is lost inside Wasm
- Flat models: 0.62–0.69
- Rule R1 reaches 1.00 recall but with 16.7% false positives
- R-GCN: 0.450

Leave-one-family-out, and the taint-lost sessions



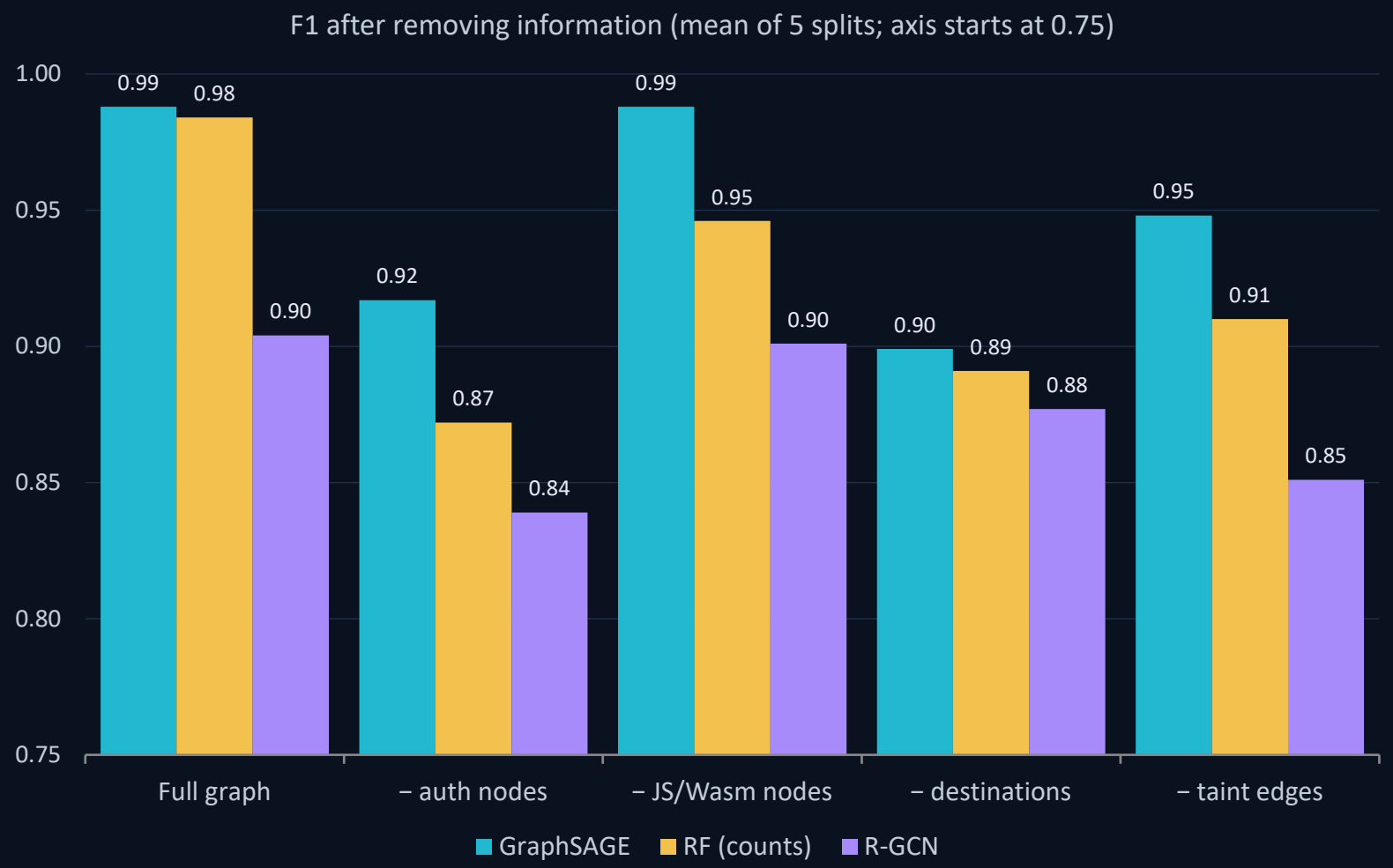
TAINT LOST INSIDE WASM

n = 25 suspicious test sessions

0%	Rule R2 (taint rule)
92%	RF (counts)
100%	GraphSAGE
64%	R-GCN

Detection rate on these sessions. A taint rule cannot see them at all.

Ablation: which information actually matters



WHAT THE DATA SAY

- Destinations matter most: GraphSAGE F1 0.988 → 0.899
- Auth nodes next: → 0.917
- Taint edges help: → 0.948, yet the structure alone keeps most of the signal
- JS/Wasm nodes barely change GraphSAGE (0.988)

The honest negative: our proposed R-GCN lost

	Parameters	Standard split F1	Unseen Wasm recall
R-GCN, untuned (v1)	349,058	0.904	0.263
R-GCN, validation-tuned (v2)	**5,498**	**0.904**	**0.450**
GraphSAGE	**28,034**	**0.988**	**0.929**

HYPOTHESES (NOT YET TESTED)

- Under-converged: on 2 of 5 splits the best epoch was 10 and 21 with train loss near 0.2
- Per-relation weights may transfer poorly to unseen relation combinations with about 350 training graphs

WHAT WE DID NOT DO

- Tune anything on the test sets
- Hide the first run: results/v1_untuned/ is kept
- Move the goalposts: the unseen-flow headline is GraphSAGE

Explainability: the path that triggered the verdict



SUSPICIOUS **confidence ≥99.3%**

550 of 550 held-out sessions in this score band were correct

HIGHLIGHTED ATTACK PATH

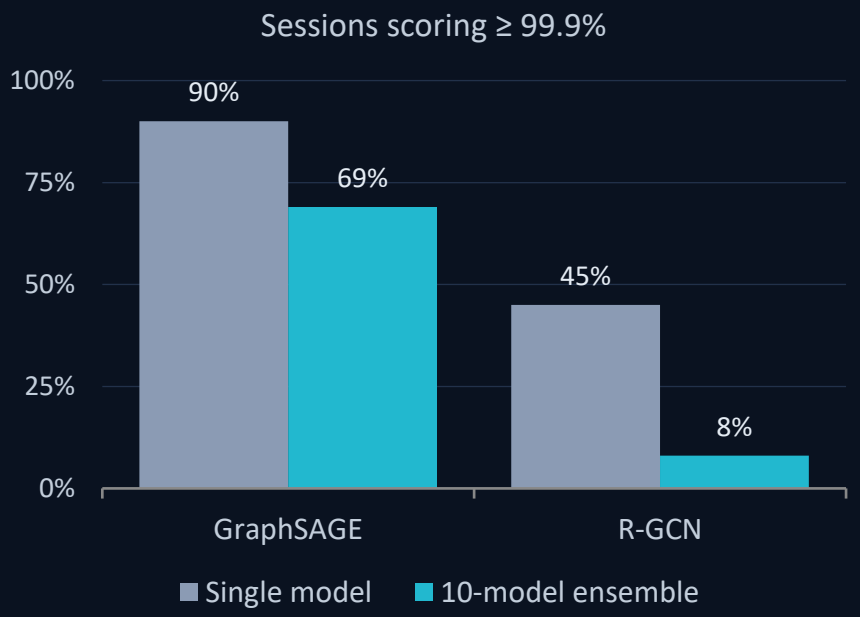
jwt_localstorage:6c988e → POST /b

stats.lab.localhost:5001

credential → request → unapproved sink

How it works. Remove one node or edge, re-score, and measure the drop in log-odds; the path is the highest-importance route from credential to domain. A saturated score falls back to a labelled structural path. **Faithfulness vs ground-truth paths: NOT YET MEASURED.**

A confidence score you can believe



Detector	Model	ECE	Log-loss	Brier
GraphSAGE	single	0.015	0.039	0.012
GraphSAGE	ensemble	0.011	0.032	0.010
R-GCN	single	0.041	0.263	0.067
R-GCN	ensemble	0.047	0.161	0.048

SUSPICIOUS

confidence ≥99.3%

Observed 100.0% correct (95% interval 99.3–100.0%): 550 of 550 held-out sessions in this score band.

The card quotes the 95% lower bound of observed accuracy. It never prints 100%.

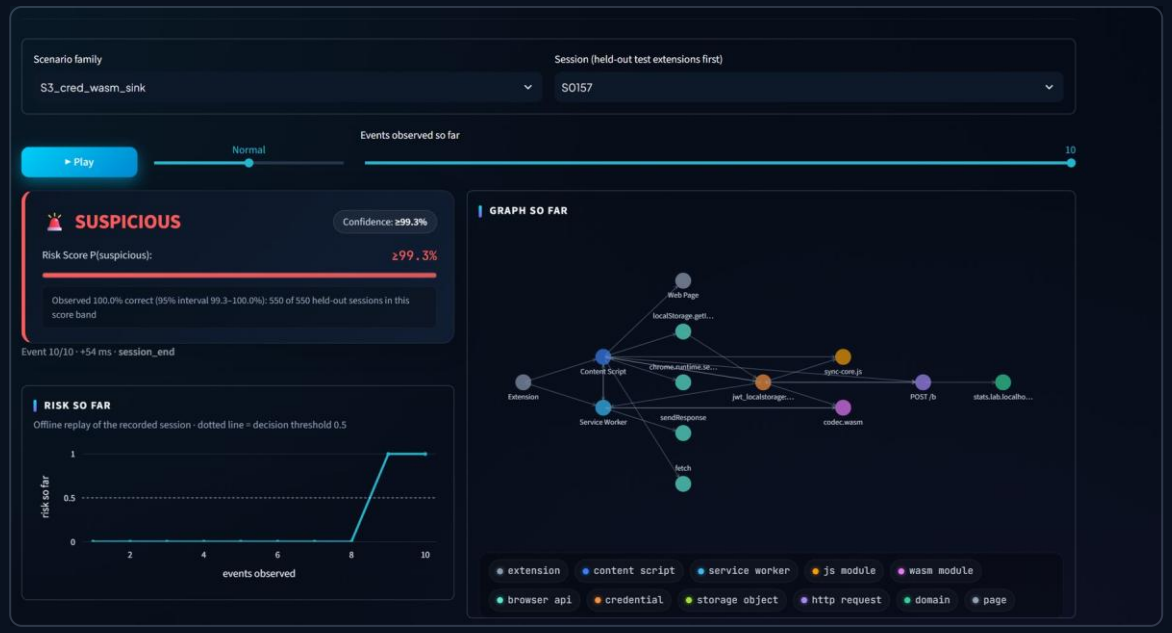
HOW IT IS BUILT

- 10 independently seeded models, scores averaged
- 576 out-of-fold predictions from unseen extensions
- Bands pooled so a higher score never shows lower confidence

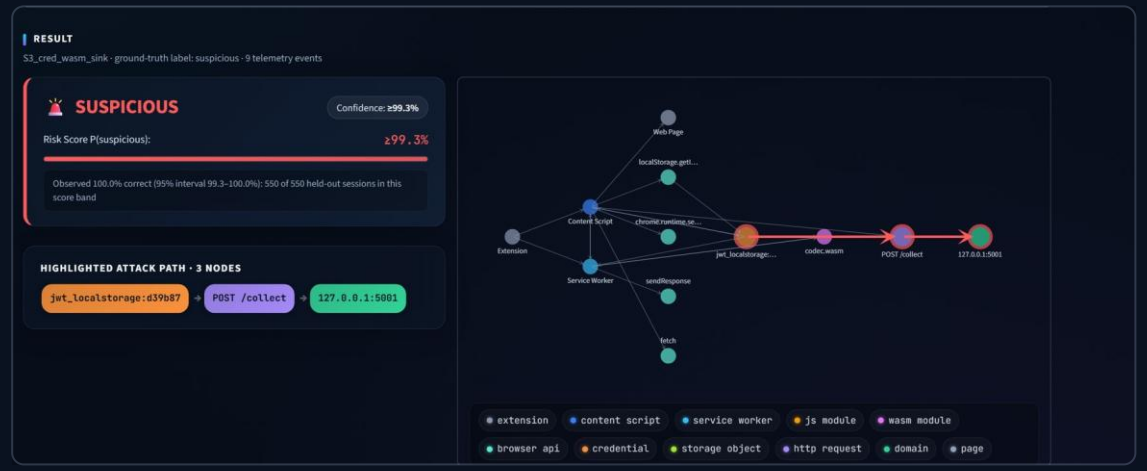
CAVEAT

On never-seen flows, R-GCN reports confidence of about 86% while being 61% accurate (ECE 0.287). Its confidence is not trustworthy there; the dashboard warns.

Live demo: event to graph to verdict, in a real browser



Session Replay: events arrive, the graph grows, the verdict appears when the session completes



Live Lab Run: a freshly generated extension in real Chromium, classified on the spot

3-minute path: 1 benign authenticated call → BENIGN 2 Wasm exfiltration → SUSPICIOUS + path 3 unseen-flow chart 4 comparison table

Limitations, and what comes next

LIMITATIONS

- Telemetry is self-reported by instrumented lab extensions, not captured at browser level
- A generator with a small step vocabulary, not real extensions
- Label defined by construction; the Wasm taint-drop rate (0.5) is our choice
- One dataset of 576 sessions; the 5 splits overlap
- No adaptive adversary
- Latency is offline CPU inference, not a deployed monitor

NEXT STEPS (NOT IMPLEMENTED)

- Browser-level capture (CDP, Arcanum-style) instead of self-reporting
- A corpus of real extensions
- Adversarial and evasion evaluation
- Understand and fix R-GCN: longer training, edge-type regularisation
- Explanation faithfulness against known flows
- Streaming, real-time deployment

Conclusion



It works end to end

Real browser events become an authentication-aware graph, a verdict, a confidence and an explained path, live.



Structure helps when instrumentation fails

GraphSAGE caught 100% of the taint-lost Wasm sessions and 0.929 of never-seen Wasm flows. A taint rule caught none of the former.



We report what lost

R-GCN did not beat GraphSAGE (0.450 vs 0.929). The untuned run, the failed calibrators and the limits are all documented.

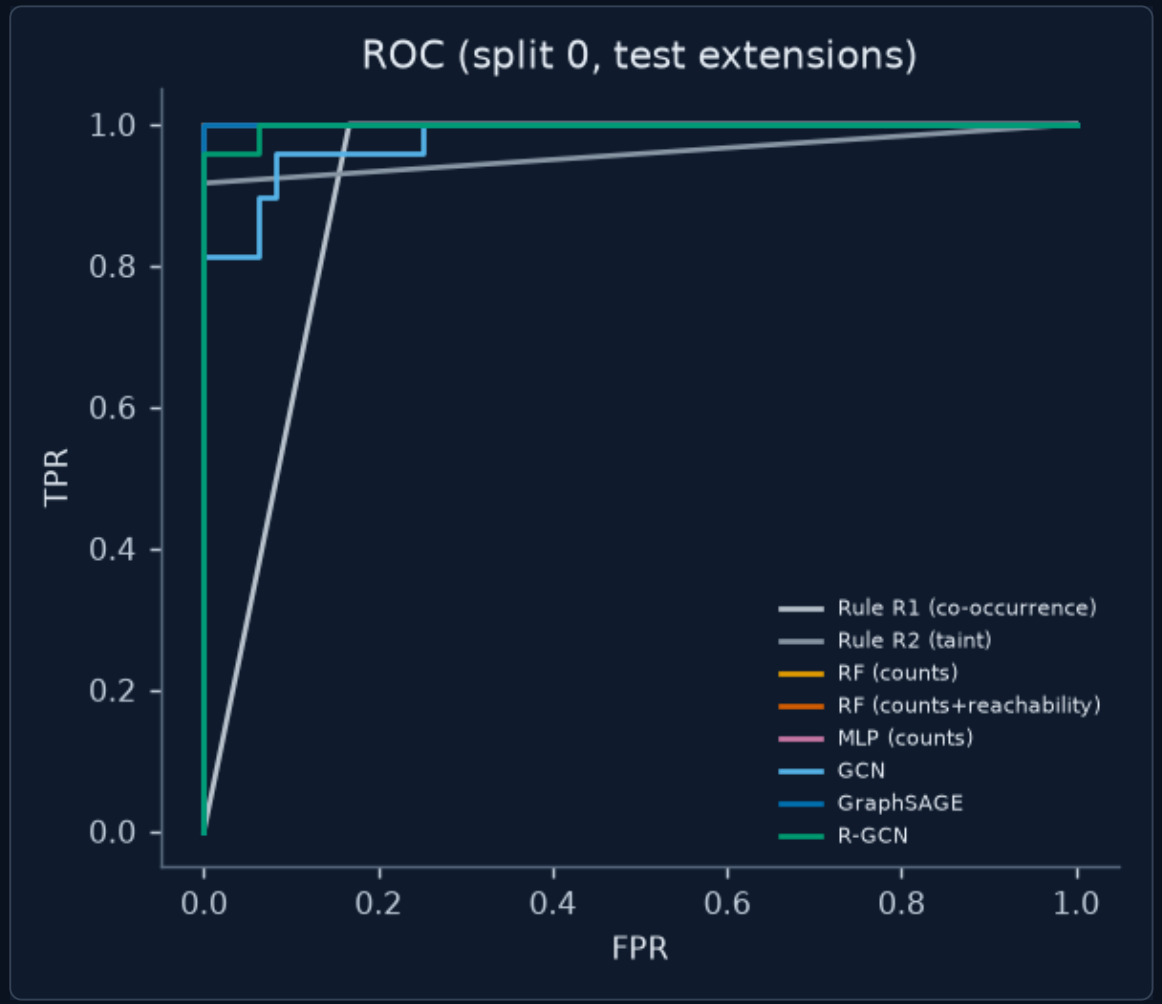
Everything shown is reproducible from the repository; every number comes from an executed run.

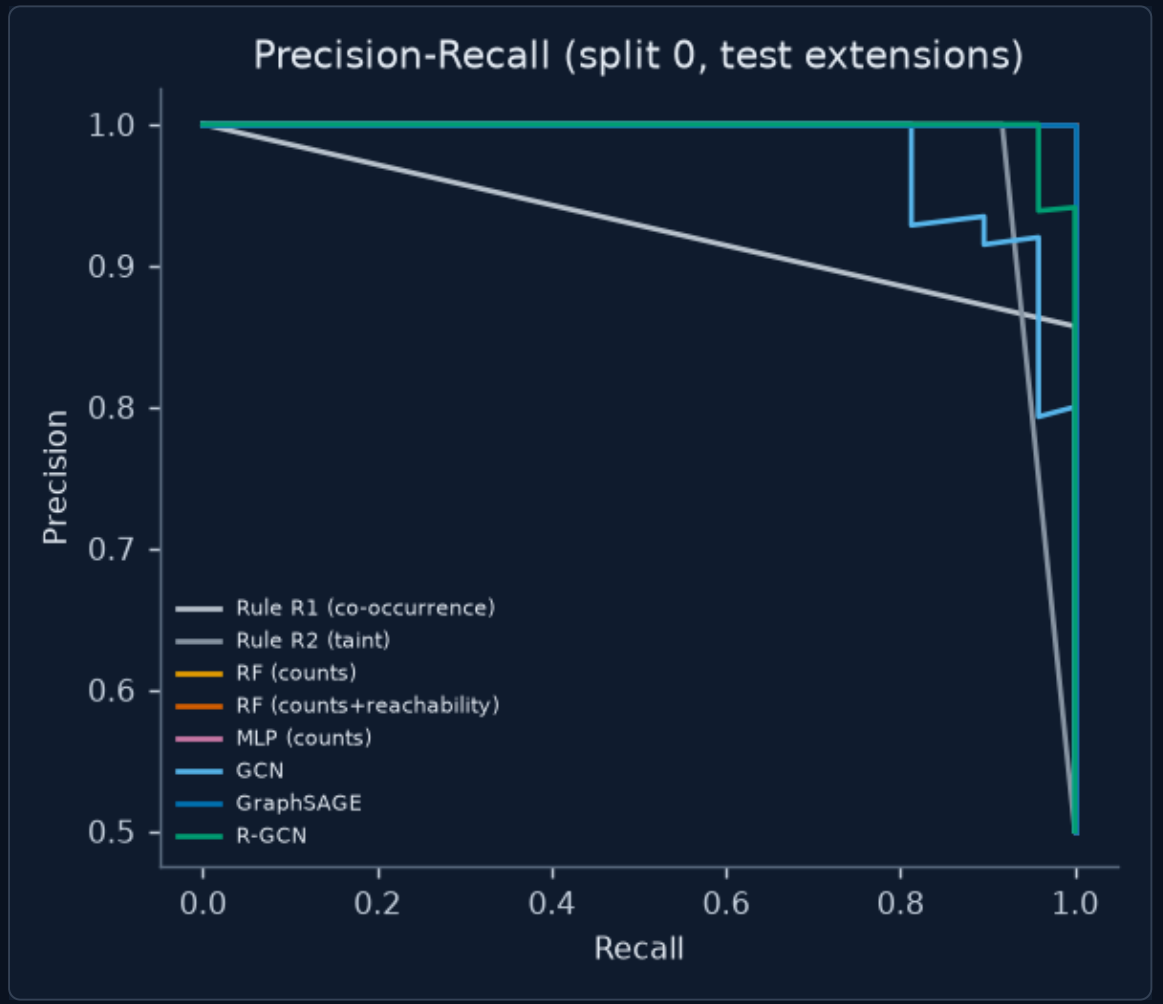
Appendix A: full results, extension-level split

Model	Precision	Recall	F1	ROC-AUC	FPR	Infer ms	Train s	Params
Rule R1 (co-occurrence)	0.857	1.000	0.923	0.917	0.167	0.004	0.00	0
Rule R2 (taint)	1.000	0.896	0.945	0.948	0.000	0.005	0.00	0
RF (counts)	0.976	0.992	0.984	0.997	0.025	0.304	0.88	7,086
RF (counts+reachability)	0.955	0.996	0.974	0.996	0.050	0.225	0.69	5,880
MLP (counts)	0.944	0.971	0.957	0.993	0.058	0.003	0.12	6,466
GCN	0.935	0.933	0.934	0.983	0.067	0.635	2.67	18,306
GraphSAGE	0.984	0.992	0.988	1.000	0.017	0.659	1.35	28,034
R-GCN	0.909	0.904	0.904	0.966	0.100	4.974	4.27	5,498

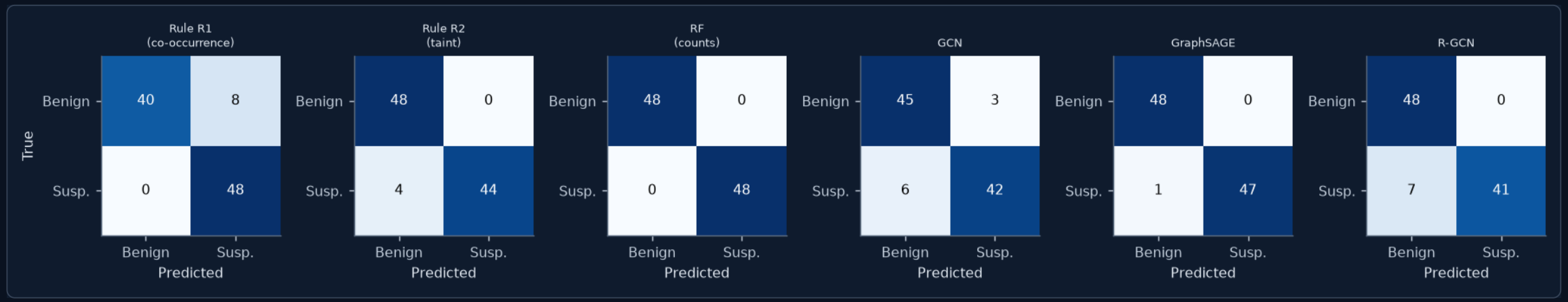
Mean over 5 extension-level splits, threshold 0.5. Inference is offline, per session graph, batch size 1, CPU; graph construction adds 0.053 ms. Params for RF are tree nodes.

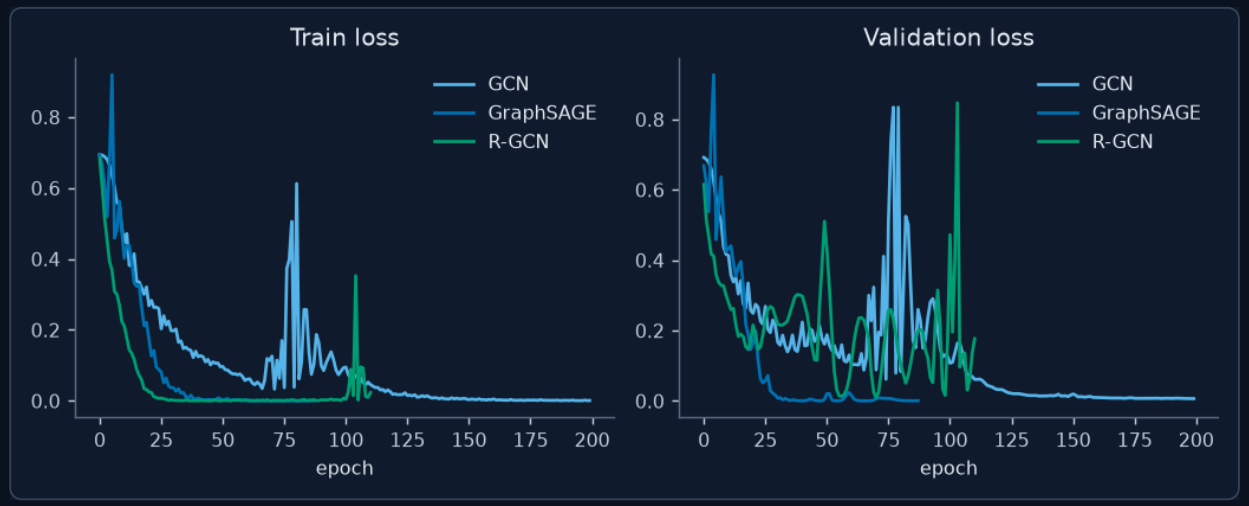
Appendix B: ROC and precision-recall curves

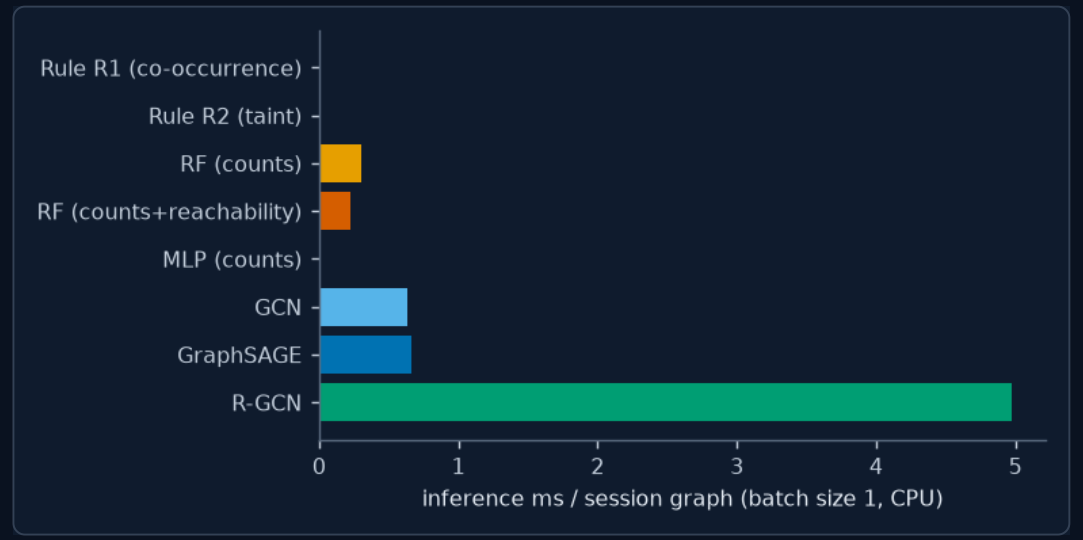




Appendix C: confusion matrices, training curves and latency







Appendix D: confidence calibration in detail

SCORE BANDS (ENSEMBLE, OUT-OF-FOLD)

Detector	Score band	Sessions	Correct	Accuracy	95% lower
GraphSAGE	50–70%	7	3	0.429	0.158
GraphSAGE	70–85%	19	15	0.789	0.567
GraphSAGE	85–100%	550	550	1.000	0.993
R-GCN	50–70%	28	14	0.500	0.326
R-GCN	70–85%	85	61	0.718	0.614
R-GCN	85–95%	132	131	0.992	0.958
R-GCN	95–100%	331	330	0.997	0.983

SAVED ENSEMBLES ON THEIR TEST SETS

Detector · training	n	Mean conf.	Accuracy	ECE
GraphSAGE · standard split	96	0.990	0.990	0.014
GraphSAGE · Wasm held out	144	0.945	0.972	0.028
R-GCN · standard split	96	0.927	0.917	0.076
R-GCN · Wasm held out	144	0.861	0.611	0.287

The verdict card quotes the 95% lower bound of the band accuracy. Adjacent bands are pooled so a higher score never shows a lower confidence. ECE above 0.2 means the confidence is not trustworthy for that detector under that shift.

Appendix E: reproducibility and what is not yet measured

RUN IT

```
./setup.sh
./run_demo.sh

streamlit run dashboard.py
```

Dataset frozen (576 graphs); re-running the lab produces a new random sample. All runs are logged in experiments.csv; the untuned v1 run is kept.

NOT YET MEASURED

- Clean-machine end-to-end re-run
- Explanation faithfulness
- Real-extension corpus
- Adversarial evaluation
- Browser-level (non-self-reported) telemetry

Appendix F: references

- Fass, Somé, Backes, Stock. DoubleX: Statically Detecting Vulnerable Data Flows in Browser Extensions at Scale. ACM CCS 2021.
- Xie, Kasi Murali, Pearce, Li. Arcanum: Detecting and Evaluating the Privacy Risks of Browser Extensions on Web Pages and Web Content. USENIX Security 2024.
- I Spy with My Little Eye: Analysis and Detection of Spying Browser Extensions. arXiv:1612.00766.
- The Blind Spot of Extension Security: WebAssembly–JavaScript Collaborative Attacks on Chrome. Electronics, doi:10.3390/electronics15143049 (title only; read before citing).
- Kipf and Welling. Semi-Supervised Classification with Graph Convolutional Networks. ICLR 2017.
- Hamilton, Ying, Leskovec. Inductive Representation Learning on Large Graphs. NeurIPS 2017.
- Schlichtkrull et al. Modeling Relational Data with Graph Convolutional Networks. ESWC 2018.